Department of Physics and Astronomy

Techniques of High-Performance Computing - PHAS0102

Week 3: GPU Programming

CPU vs GPU

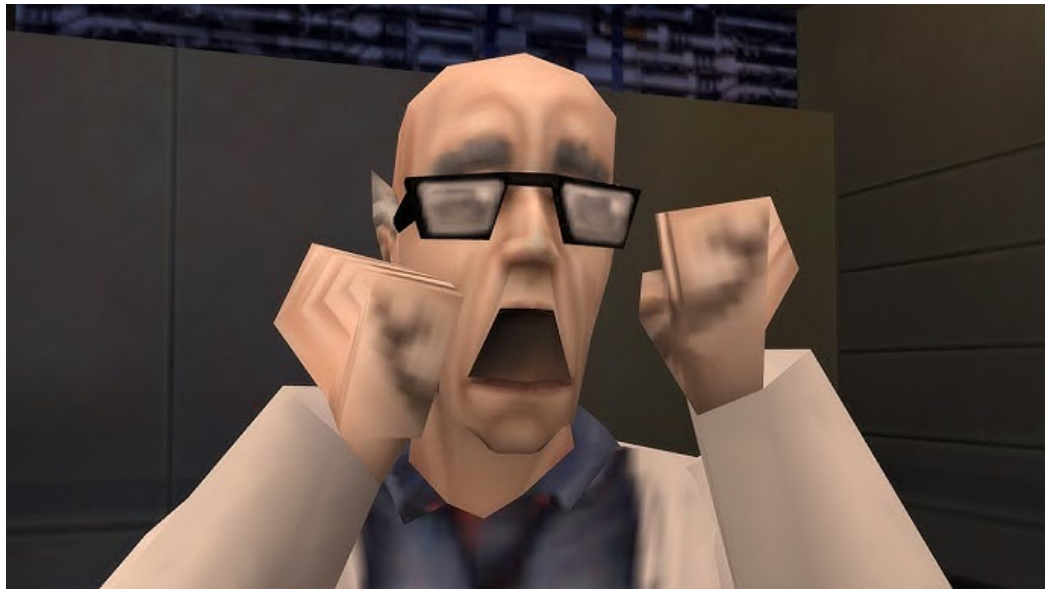
- CPU = Central Processing Unit
  - Designed to be good at everything a computer needs to do.
- GPU = Graphical Processing Units
  - Designed to be good at processing 3D Graphics

3D Graphics

- Perform same operation on millions of data points
  - Matrix transform of millions of vertices
  - Texture mapping millions of polygons
  - Lighting and color calculation of millions of pixels
- Most of the work can be done in parallel
  - Prioritize parallelism over single core speed
  - Better to do 1000 jobs at the same time in reasonable per-job time rather than 1 job really quickly.

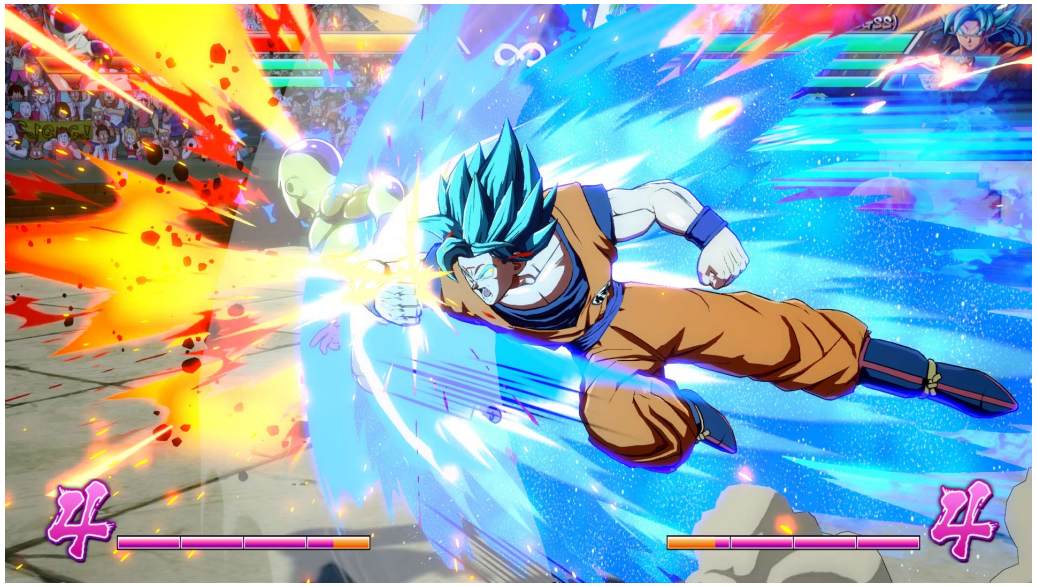
GPU history

- First GPUs used fixed-function pipelines
  - Vertex transform, lighting, texturing were fixed
  - Why games in the late 90s, early 2000s looked similar



GPU history

- Introduction of GPU “shaders”
  - Small pieces of code can be run
  - Programmable graphics pipeline



GPGPU Programming

- *General Purpose GPU Programming*
- Exploit GPU hardware to do non-graphics work
  - Early GPGPU transformed problems into graphics work
- Modern day libraries now allow you to use the shader units directly
  - CUDA
  - ROCm
  - DirectX 12+
  - Vulcan

Linear Algebra Operators for GPU Implementation of Numerical Algorithms

Jens Krüger and Rüdiger Westermann
Computer Graphics and Visualization Group, Technical University Munich*

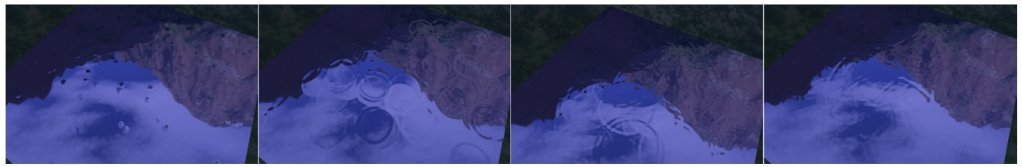
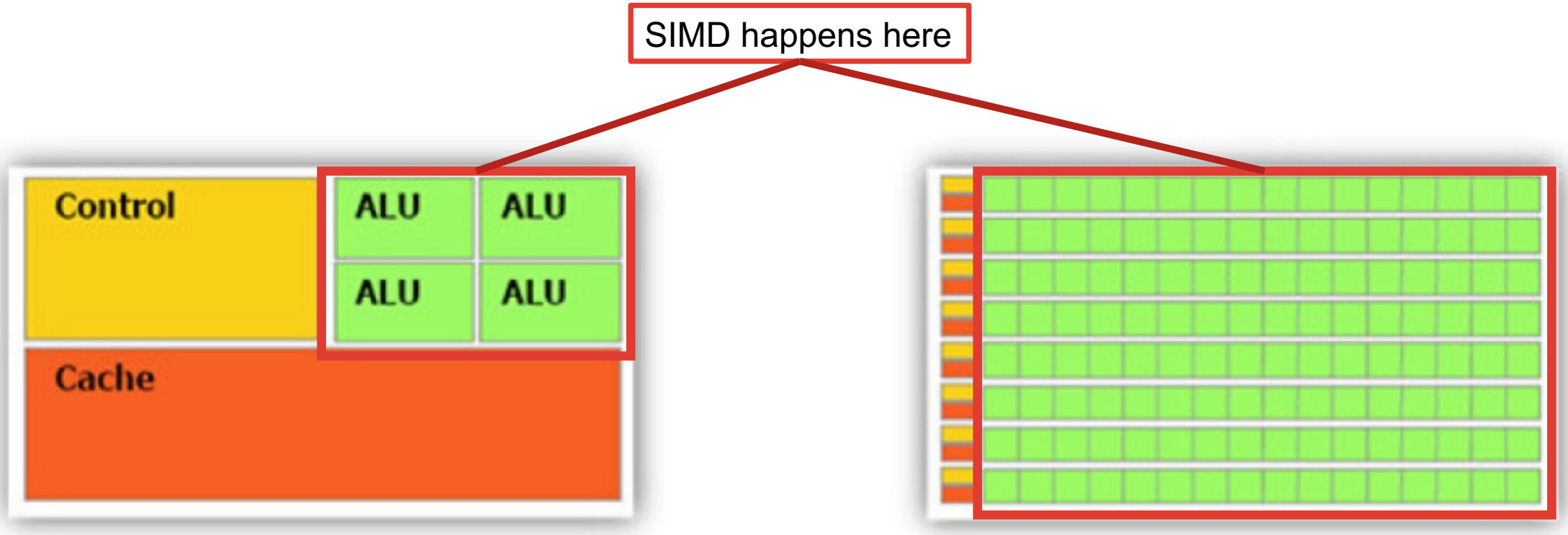


Figure 1: *We present implementations of techniques for solving sets of algebraic equations on graphics hardware. In this way, numerical simulation and rendering of real-world phenomena, like 2D water surfaces in the shown example, can be achieved at interactive rates.*

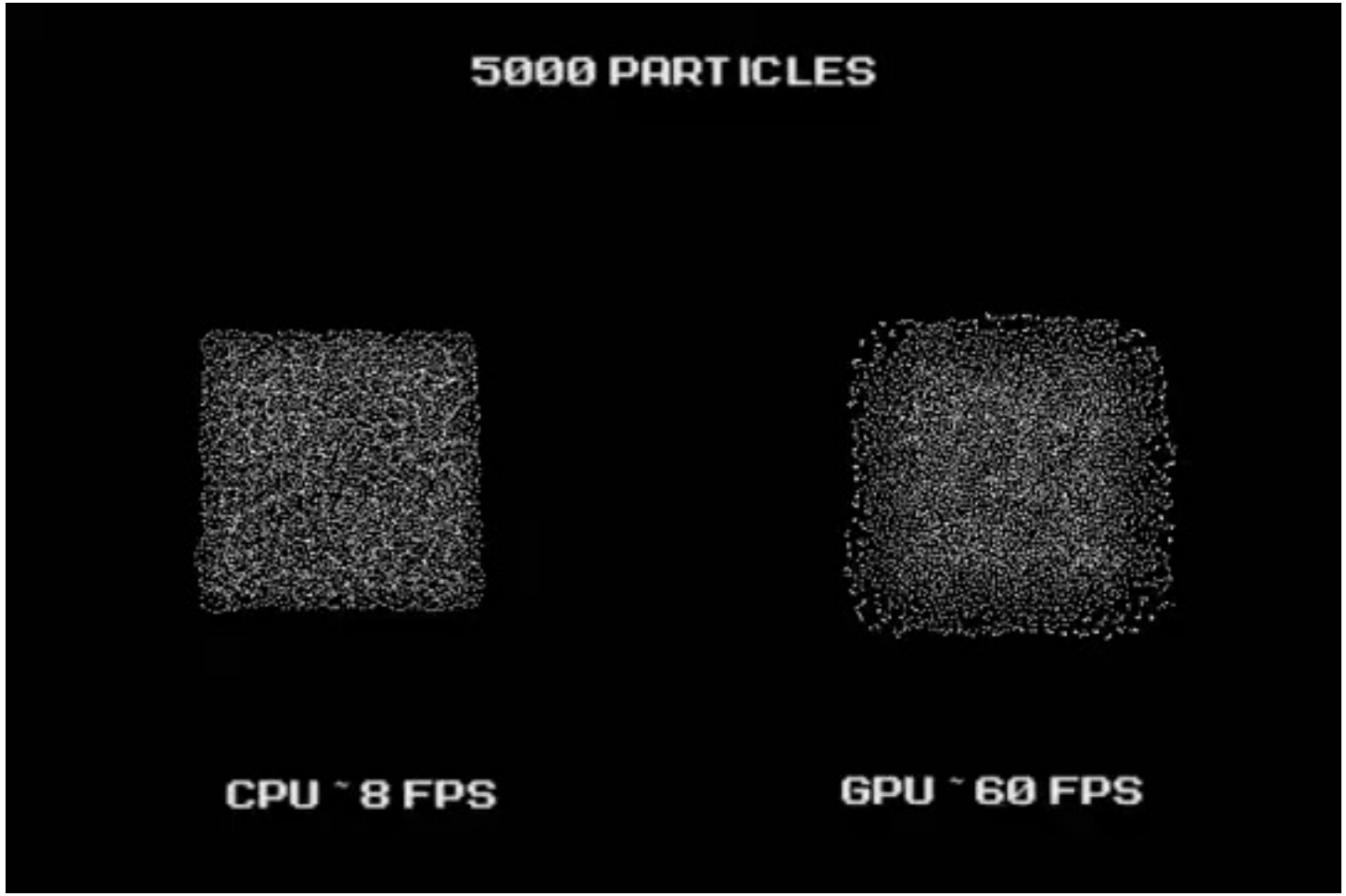
CPU vs GPU Architecture



CPU Core Speed: 3.0+ GHz

GPU Core Speed: 1 – 2 GHz

CPU vs GPU



GPU Programming

- CUDA
  - Most popular and mature development platform.
  - Support for C/C++, Fortran, Python etc.
  - nVidia GPUs only
  - Proprietary
- ZLUDA
  - CUDA for non-nVidia GPUs
  - Open Source
  - “Drop-In”
  - Killed twice already, first by nVidia, then AMD
- HIP
  - AMDs version of CUDA
  - Compile CUDA code for different GPUs
  - Sort of works?
- Kompute
  - Vulkan development
  - Geared towards ML
- OpenCL
  - Dead

CUDA with Numba

CUDA Terms

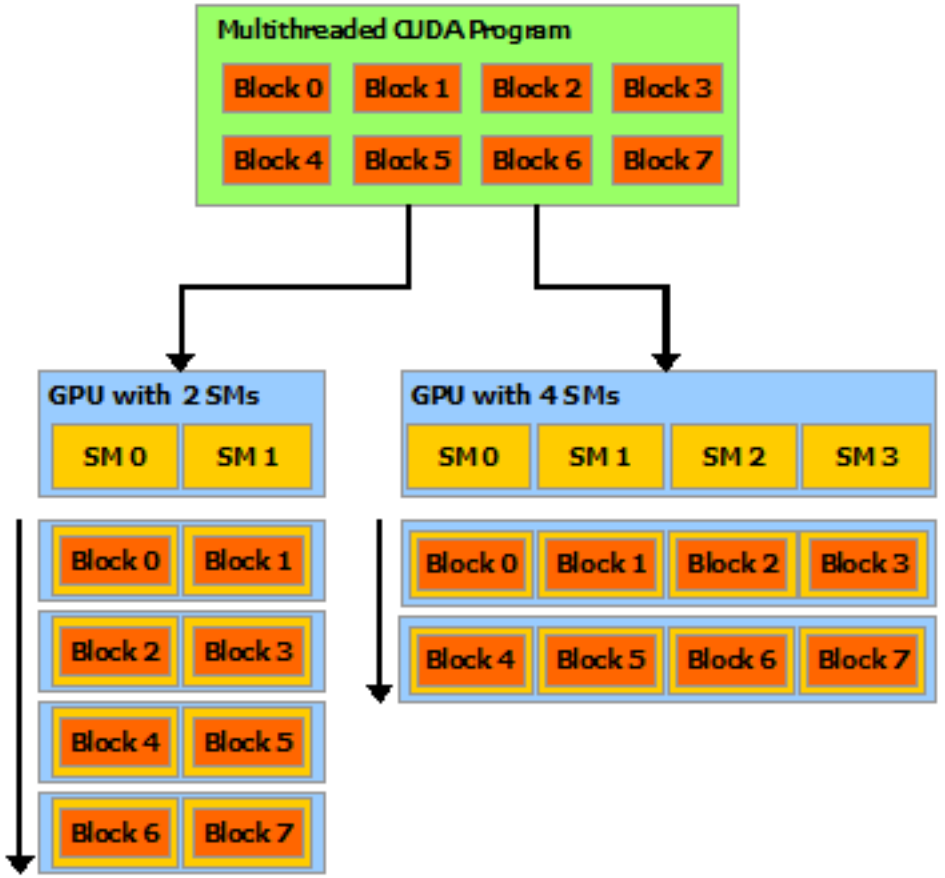
- Host
  - The machine that has the GPU
- Device
  - The GPU
- Kernel
  - The function launched from the host on the GPU
- Device function
  - A function that can only be called from a kernel

CUDA Threading Model

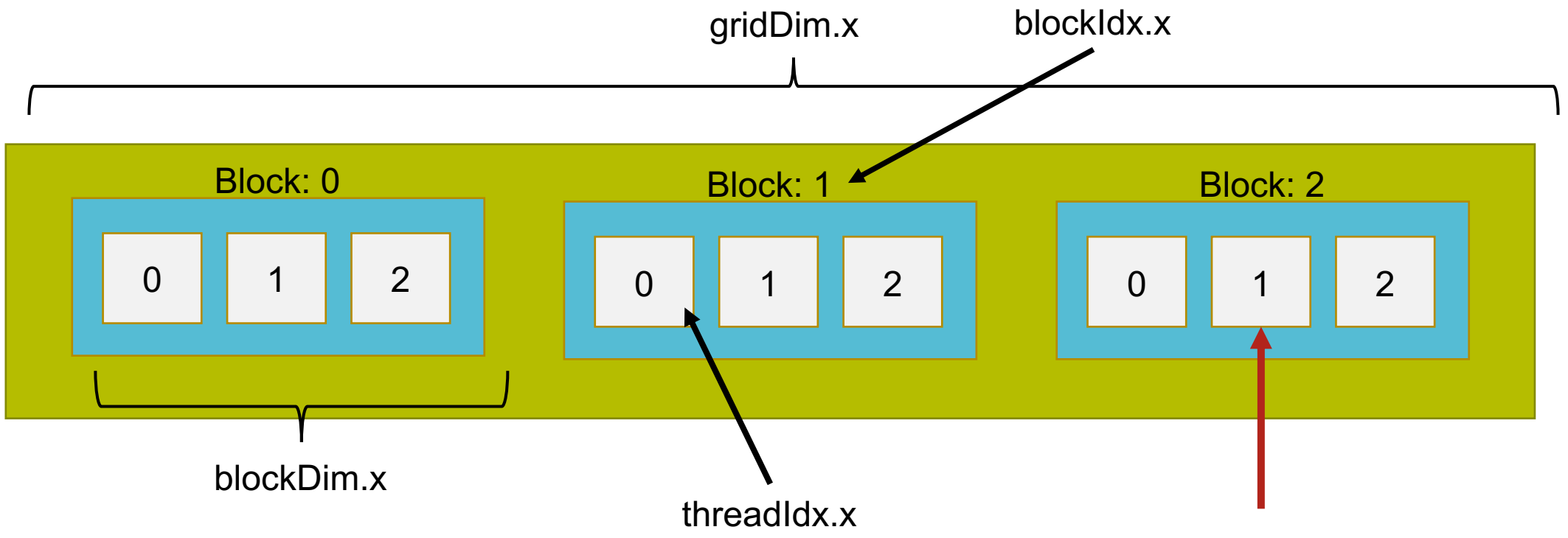
- Streaming multiprocessors
  - True GPU cores.
  - A100 has 108 of them
- Warps
  - A scheduled collection of executing threads
  - 32 threads
  - Same execution path
- Block
  - A collection of threads
- Threads
  - The computing unit

CUDA scheduling

- CUDA will schedule blocks to an SM
- Blocks are then split into warps of threads
  - 32-64 threads at a time
- CUDA kernel is executed until block is exhausted of threads
- New block scheduled on SM
- Repeat until all blocks exhausted.



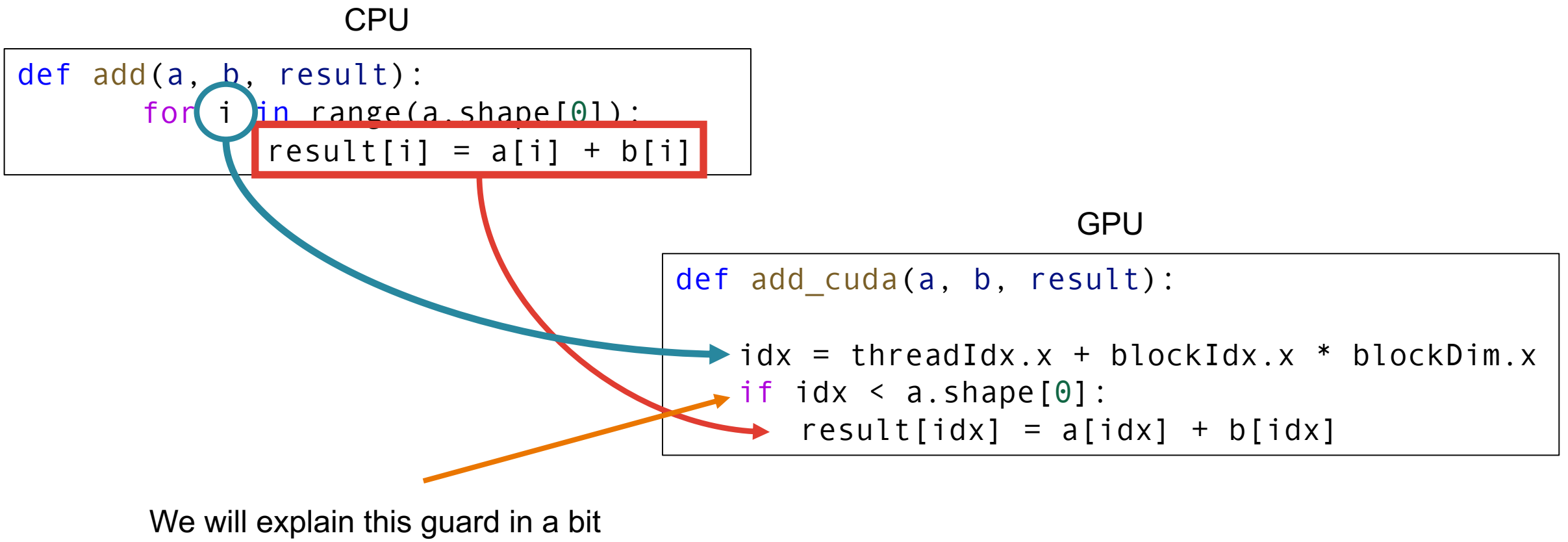
CUDA Threading model



$$index_{abs} = blockDim * blockIdx + threadIdx$$

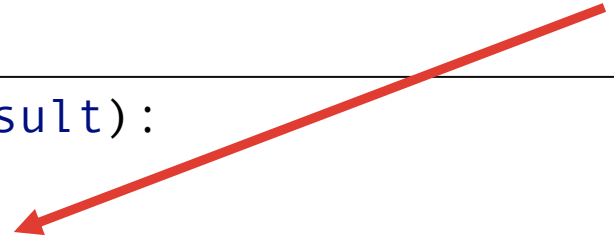
$$index_{abs} = \quad 3 \quad * \quad 2 \quad + \quad 1 \quad = \quad 7$$

CUDA Threading model



CUDA Threading model

Numba convenience function.



```
def add_cuda(a, b, result):

    idx = cuda.grid(1)
    if idx < a.shape[0]:
        result[idx] = a[idx] + b[idx]
```

CUDA Threading model

- Same mechanisms for 2D and 3D problems

```
def matrix_product(mat1, mat2, result):
    for c in range(mat2.shape[1]):
        for r in range(mat1.shape[0]):
            value = 0
            for i in range(mat1.shape[1]):
                value += mat1[r, i] * mat2[i, c]
            result[r,c] = value
```

```
def matrix_product_cuda(mat1, mat2, result):
    r, c = cuda.grid(2)
    if r < result.shape[0] and j < result.shape[1]:
        value = 0.
        for k in range(mat1.shape[1]):
            value += mat1[i, k] * mat2[k, j]
        result[r, c] = value
```

Invoking kernels

- Kernels are invoked by specifying the number of blocks and threads per block.
- Numba tries to match CUDA-C convention

CUDA-C: `add_cuda<<<griddim, blockdim>>>(a2, b2, r2)`

Numba: `add_cuda[griddim, blockdim](a2, b2, r2)`

Kernel name Launch Specifiers Function arguments

1D Example: Vector add

- Maximum of 1024 threads per block
  - With threads alone we can only handle problems efficiently of this size
- Thread numbers should be multiple of 32.
- Spawn appropriate number of blocks to cover problem size

```
@cuda.jit
def add_cuda(a, b, result):

    idx = cuda.grid(1)
    if idx < a.shape[0]:
        result[idx] = a[idx] + b[idx]
```

Rule of Thumb:

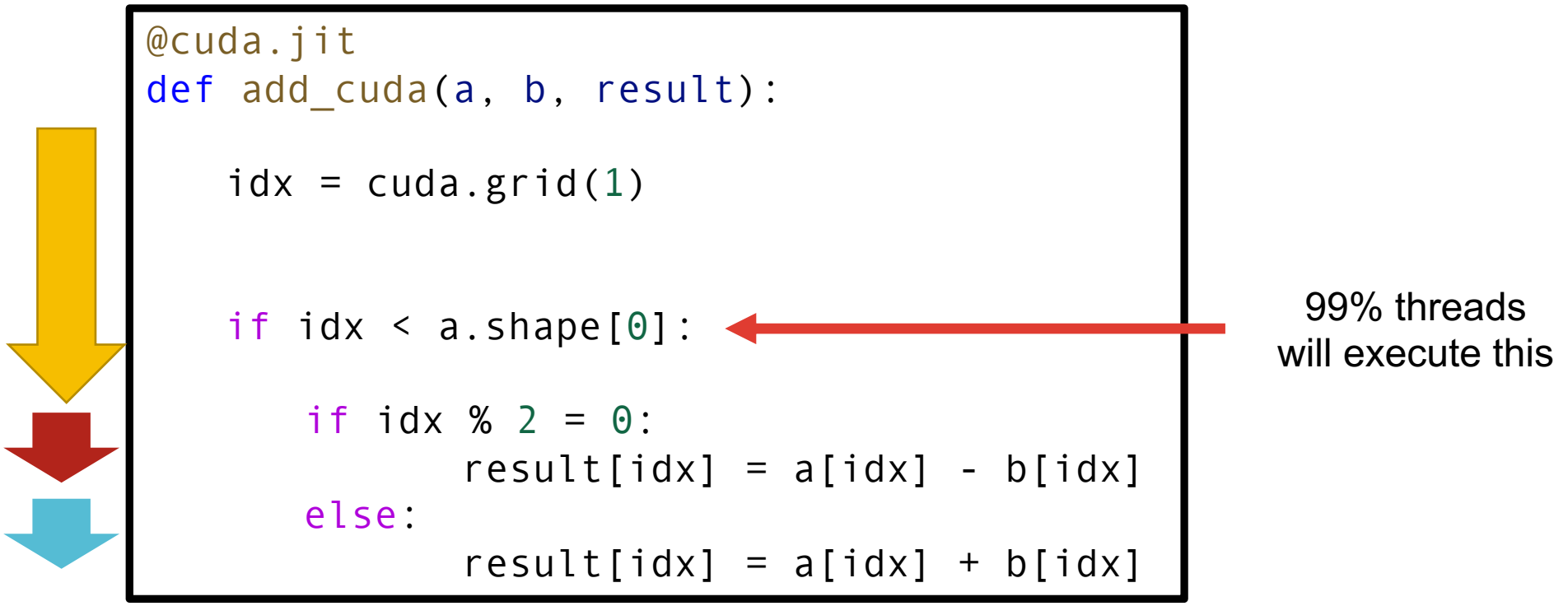
$$N_{blocks} = \left\lceil \frac{N}{N_{threads}} \right\rceil$$

```
import math
# Array size
N = a.size
# Launch specifiers
blockdim = 1024
griddim = int(math.ceil(N / blockdim))
# Launch kernel
add_cuda[griddim, blockdim](a, b, r2)
```

$N = 1{,}000{,}000$
$N_{threads} = 1024$
$N_{blocks} = 977$
$N_{threads} * N_{blocks} = 1{,}000{,}448$

Warps

- Why 32? Because threads in blocks are scheduled in warps which are either in groups of 32 or 64
- At each line 32 threads are running in parallel
- Must follow same execution path
  - Branches can change the path!!!!
- If path is changed then the execution is serial



Compute throughput has been cut in half!

2D Example: Matrix Multiply

- We can split our threads across dimensions
  - Threads must not exceed 1024
  - Like memory, it's the same as 1D but reinterpreted
- Pass the parameters as tuples
- Thread variables will have y parameter
  - `cuda.threadIdx.y`
  - `cuda.blockDim.y`
- Same rule of thumb for each dimension

```
@cuda.jit
def matrix_product_cuda(mat1, mat2, result):
    r, c = cuda.grid(2)
    if r < result.shape[0] and j < result.shape[1]:
        value = 0.
        for k in range(mat1.shape[1]):
            value += mat1[i, k] * mat2[k, j]
        result[r, c] = value
```

```
N, M = result.shape
blockdim = (16, 16) # 16 * 16 = 256 threads
nblocks_x = int(math.ceil(N / blockdim[0]))
nblocks_y = int(math.ceil(M / blockdim[0]))
griddim =(nblocks_x, nblocks_y)

matrix_product_cuda[griddim,blockdim](mat1, mat2, result)
```

Synchronization

- When we invoke a function. It will immediately ‘finish’
- This is because calls are ‘non-blocking’.
  - ‘Non-blocking refers to functions that immediately return control
- Remember the CPU isn’t being run! So no need to wait
  - Can run other CPU code in the meantime
- We need to explicity wait if we want the result
  - We can use the function `cuda.synchronize()`

```
%%timeit -n1 -r1

add_better[griddim, blockdim](a2, b2, r2)
```

```
316 µs ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

```
%%timeit -n1 -r1

add_better[griddim, blockdim](a2, b2, r2)
cuda.synchronize()
```

```
9.05 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

Threads Synchronization

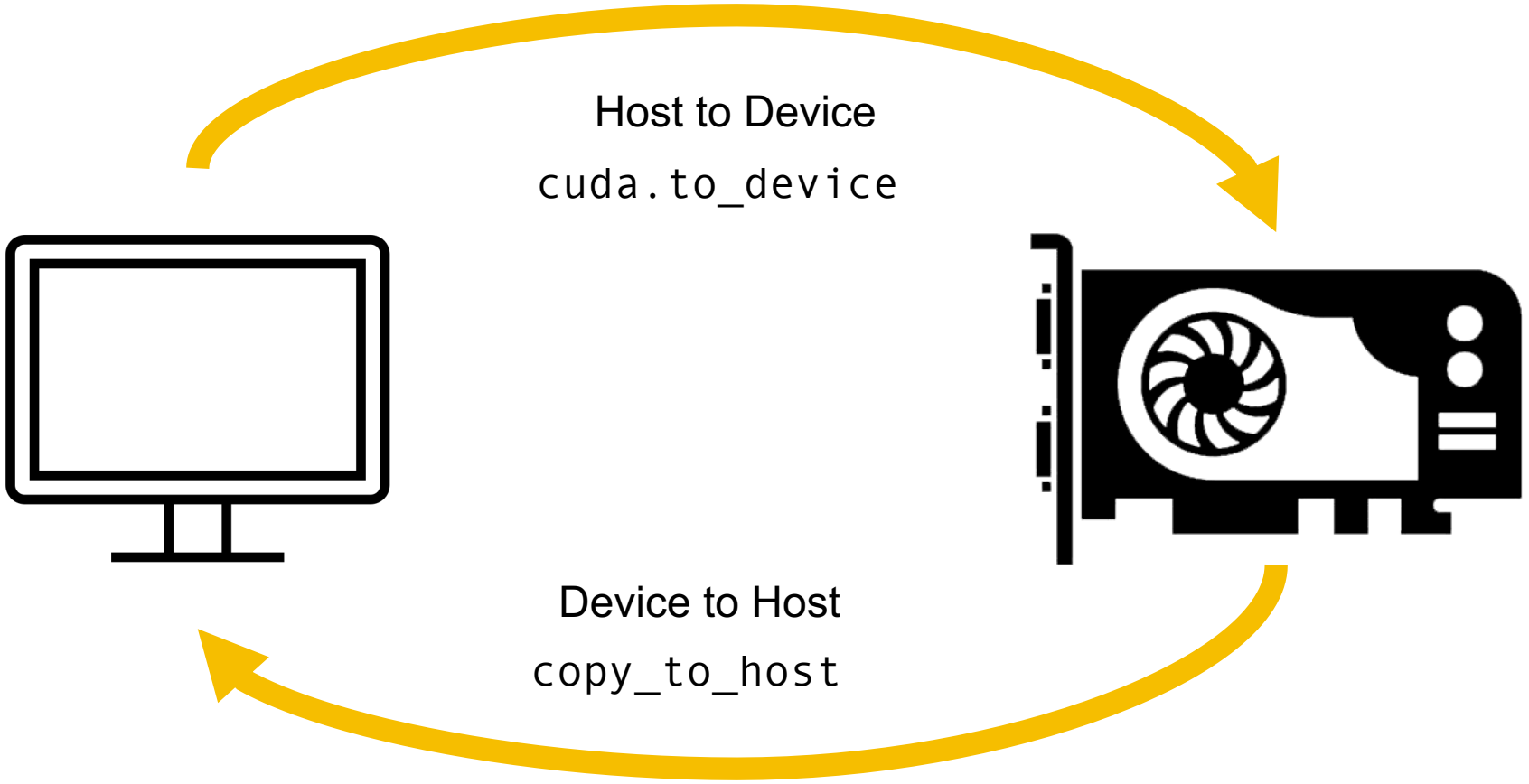
- You can also control thread movement
- Create a barrier that forces all threads in a block to synchronize
- `cuda.sync_threads()`
- We will examine its usage later

CUDA Memory

Moving data

- The CPU and GPU are physically separate
- This requires us to move data around
- Scalars (floats, int etc) can be passed in directly
- Arrays must be explicitly transferred

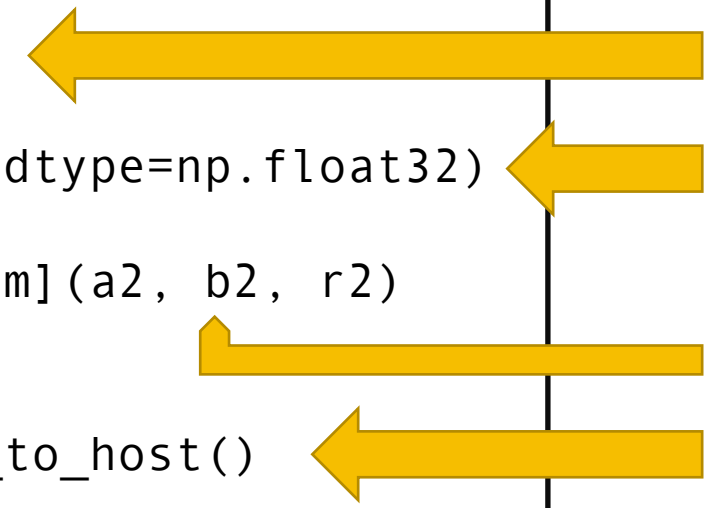
Moving data



Memory movement

```
N = 100_000_000

a = np.random.rand(N).astype("float32")
b = np.random.rand(N).astype("float32")

a2 = cuda.to_device(a)
b2 = cuda.to_device(b)
r2 = cuda.device_array(N,dtype=np.float32)

add_cuda[griddim, blockdim](a2, b2, r2)


result_fromgpu = r2.copy_to_host()

print(result_gpu)
```

```
@cuda.jit
def add_cuda(a, b, result):
    idx = cuda.grid(1)
    if idx < a.shape[0]:
        result[idx] = a[idx] + b[idx]
```



Transfer to GPU

Create array directly on GPU

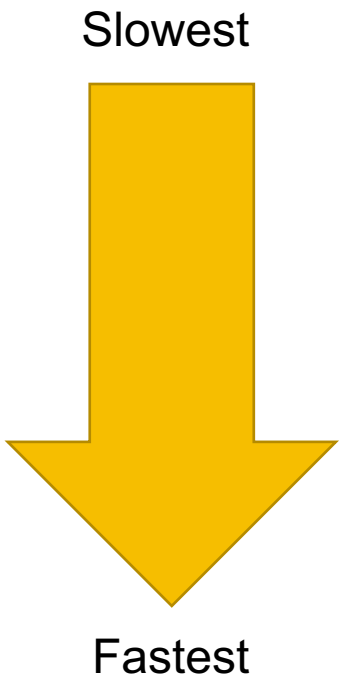
Device arrays passed into kernel

Copy result back to CPU

Copy_to_host is ‘blocking’ so it automatically synchronises

CUDA Device memory Model

- 3-tier model



- Global Memory
  - Data stored in GPU RAM
  - Shared between kernels and threads
- Shared Memory
  - Temporary cache
  - Shared between threads in block
- Registers
  - Variables in kernel
  - Per Thread

Global memory

- Resides in GPU RAM
- Slowest access time
- Where data from host transfer will reside
- Gigabytes of storage

We can't return an array
so it must be passed in

```
@cuda.jit
def add_cuda(a, b, result):
    idx = cuda.grid(1)
    if idx < a.shape[0]:
        result[idx] = a[idx] + b[idx]
```

- Array arguments must reside in Global memory
- Scalar arguments do not need to be transferred

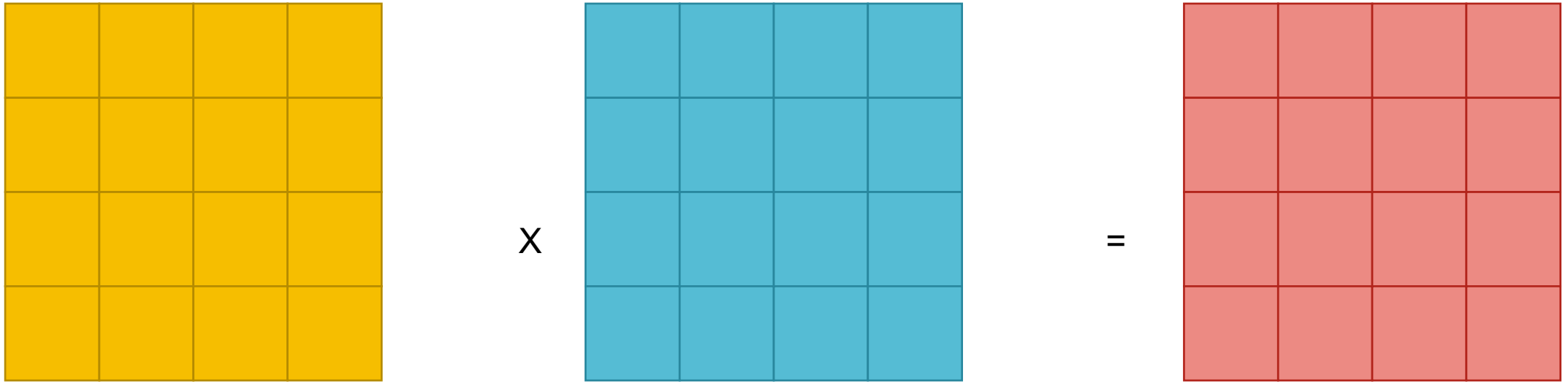
Accessed like normal array

Shared Memory

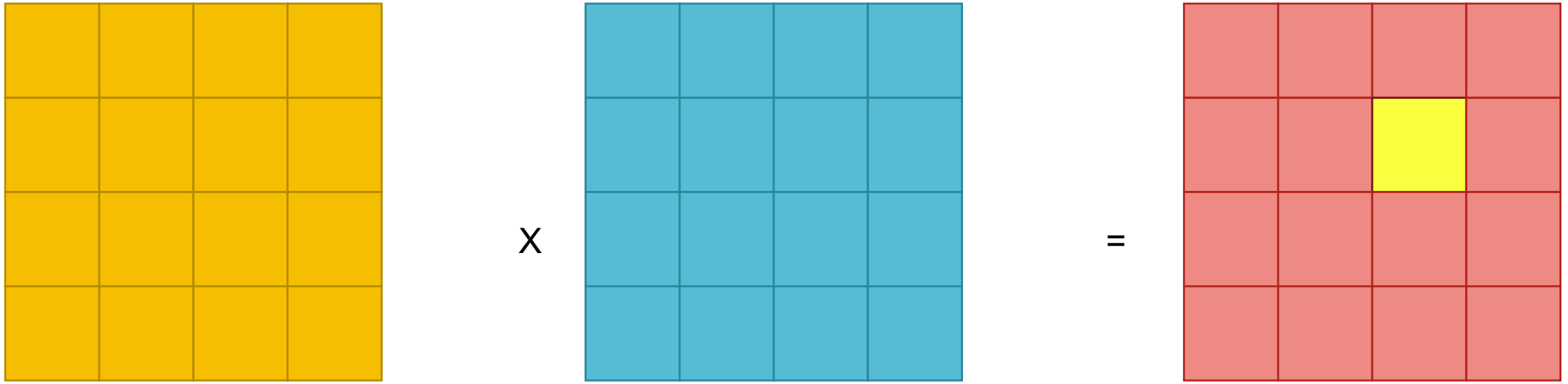
- On-chip memory
- Cache shared by threads in a block
- Seen as ‘array’ in the thread
- Allows for caching reusable data in computation
- Allows sharing data between threads!
- Either statically determined or dynamically

```
sA = cuda.shared.array(shape=(10,), dtype=float32)
```

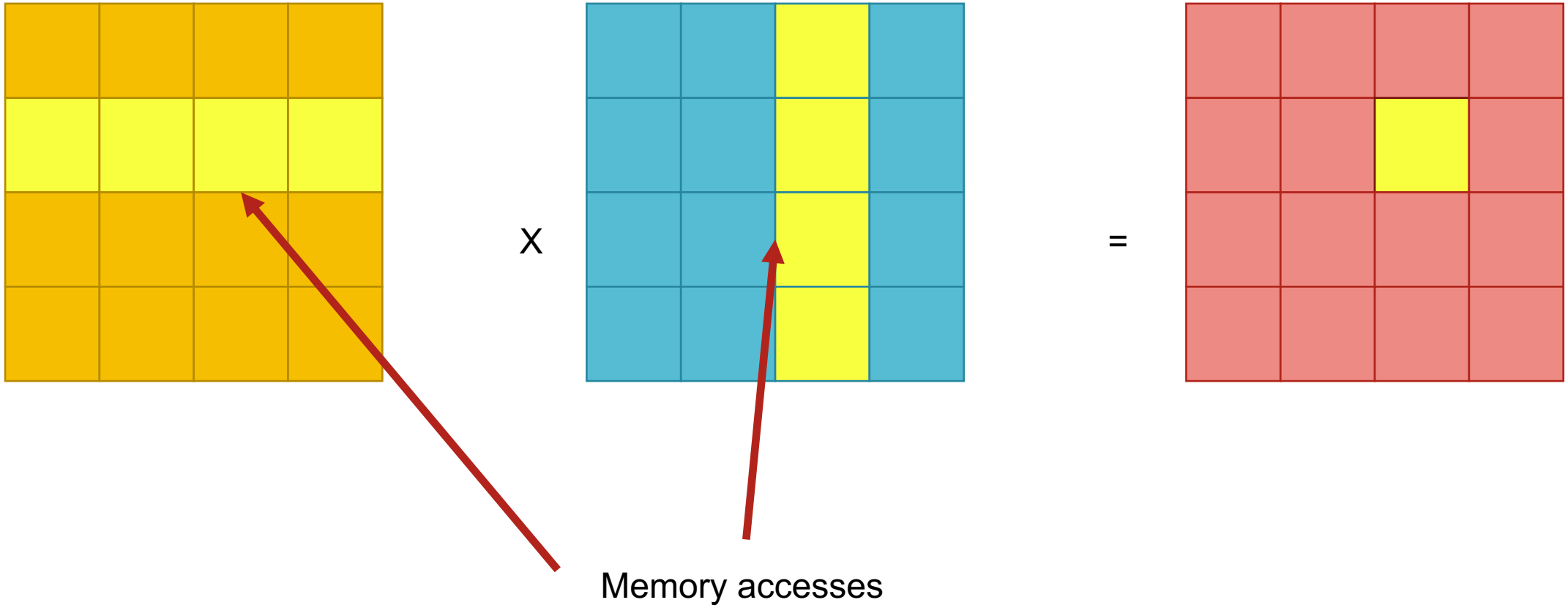
Shared Memory example



Shared Memory example



Shared Memory example



```
@cuda.jit
def matmul(A, B, C):
"""Perform square matrix multiplication of
C = A * B."""
    i, j = cuda.grid(2)
    if i < C.shape[0] and j < C.shape[1]:
        tmp = 0.
        for k in range(A.shape[1]):
            tmp += A[i, k] * B[k, j]
            C[i, j] = tmp
```

```
%timeit x_h @ y_h
```

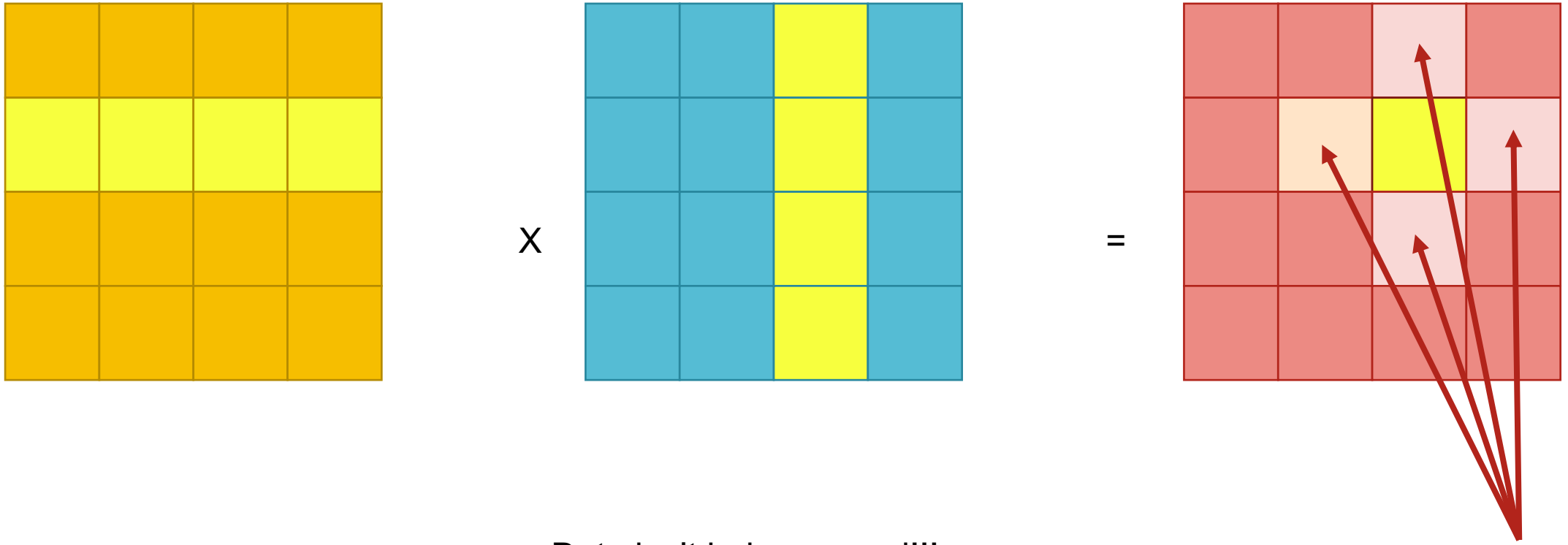
```
44.7 ms ± 8.26 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

```
[32] %%timeit
     matmul[blockspergrid, threadsperblock](x_d, y_d, z_d)
     cuda.synchronize()
```

OK but not amazing

```
18.8 ms ± 261 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

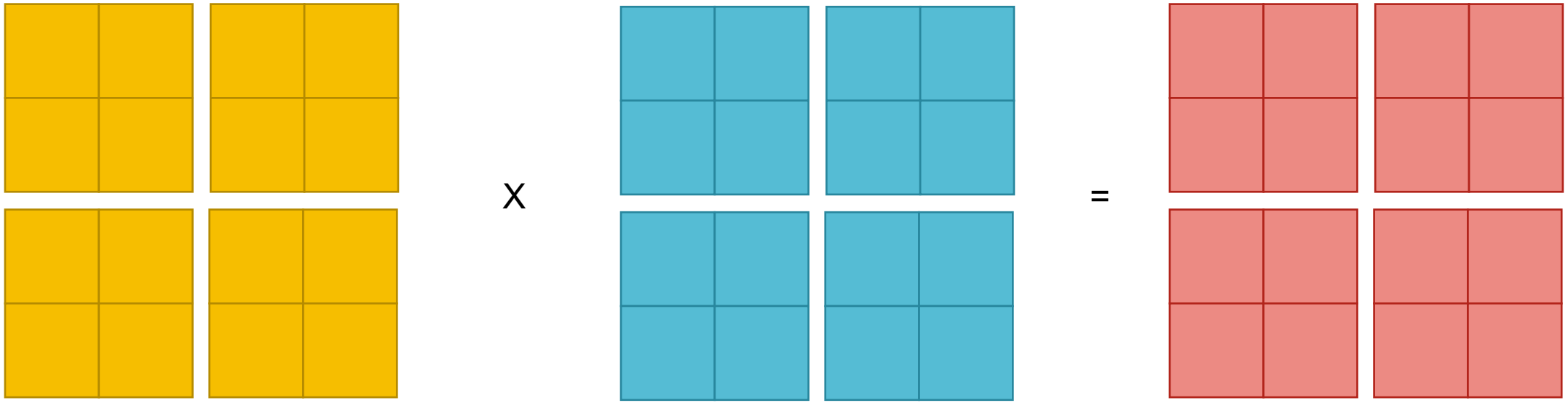
Shared Memory example



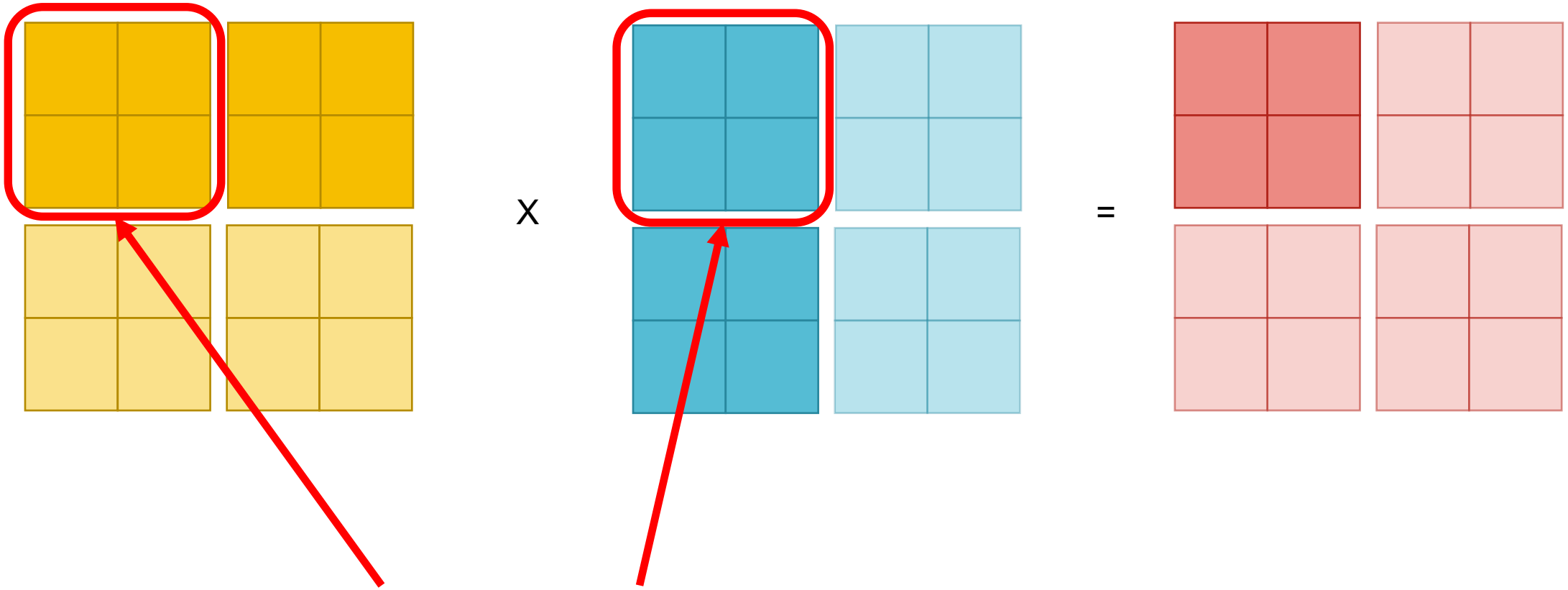
Data isn't being reused!!!

Similar accesses

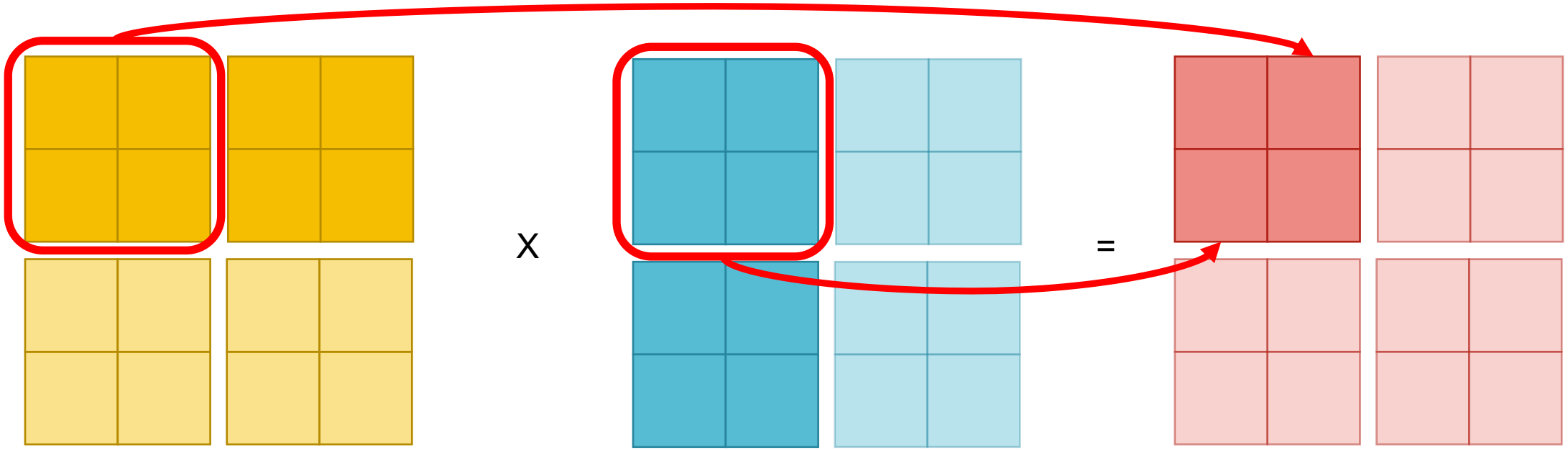
Shared Memory example



Shared Memory example

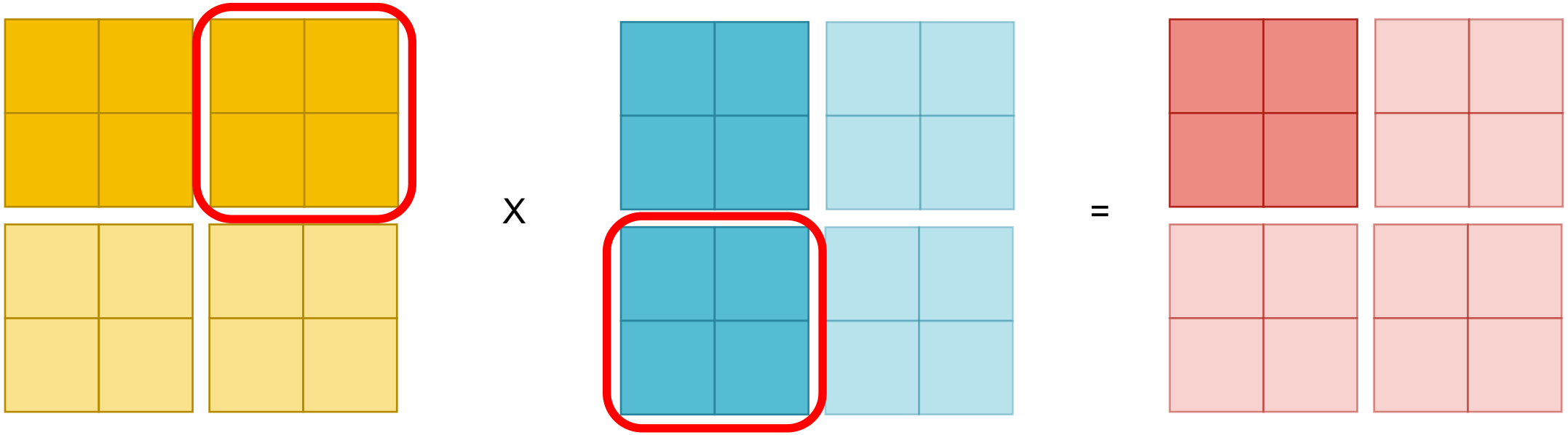


Shared Memory example



Thread block computes partial matrix multiply using cache

Shared Memory example



Thread block moves to next set and repeats

```
TPB = 16

@cuda.jit
def fast_matmul(A, B, C):
    sA = cuda.shared.array(shape=(TPB, TPB), dtype=float32)
    sB = cuda.shared.array(shape=(TPB, TPB), dtype=float32)

    x, y = cuda.grid(2)

    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y
    bpg = cuda.gridDim.x
    tmp = float32(0.)

    for i in range(bpg):

        sA[ty, tx] = 0
        sB[ty, tx] = 0
        if y < A.shape[0] and (tx + i * TPB) < A.shape[1]:
```

Create 16x16 Shared memory arrays for A and B tiles

Get block and grid positions

```
for i in range(bpg):

    sA[ty, tx] = 0
    sB[ty, tx] = 0
    if y < A.shape[0] and (tx + i * TPB) < A.shape[1]:
        sA[ty, tx] = A[y, tx + i * TPB]
    if x < B.shape[1] and (ty + i * TPB) < B.shape[0]:
        sB[ty, tx] = B[ty + i * TPB, x]

    # Wait until all threads finish preloading
    cuda.syncthreads()

    # Computes partial product on the shared memory
    for j in range(TPB):
        tmp += sA[ty, j] * sB[j, tx]

    # Wait until all threads finish computing
    cuda.syncthreads()

if y < C.shape[0] and x < C.shape[1]:
```

Loop through matrix in blocks

Clear cache

load columns from A into cache

load rows from B into cache

Wait until all threads finished loading

Compute partial mat-mul

Wait until matrix computation is complete

```
    if y < A.shape[0] and (tx + i * TPB) < A.shape[1]:
        sA[ty, tx] = A[y, tx + i * TPB]
    if x < B.shape[1] and (ty + i * TPB) < B.shape[0]:
        sB[ty, tx] = B[ty + i * TPB, x]

    # Wait until all threads finish preloading
    cuda.syncthreads()

    # Computes partial product on the shared memory
    for j in range(TPB):
        tmp += sA[ty, j] * sB[j, tx]

    # Wait until all threads finish computing
    cuda.syncthreads()

if y < C.shape[0] and x < C.shape[1]:
    C[y, x] = tmp
```

Store matrix result

Result

```
%timeit x_h @ y_h
```

```
44.7 ms ± 8.26 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

```
[32] %%timeit
     matmul[blockspergrid, threadsperblock](x_d, y_d, z_d)
     cuda.synchronize()
```

```
18.8 ms ± 261 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

```
[30] %%timeit
     fast_matmul[blockspergrid, threadsperblock](x_d, y_d, z_d)
     cuda.synchronize()
```

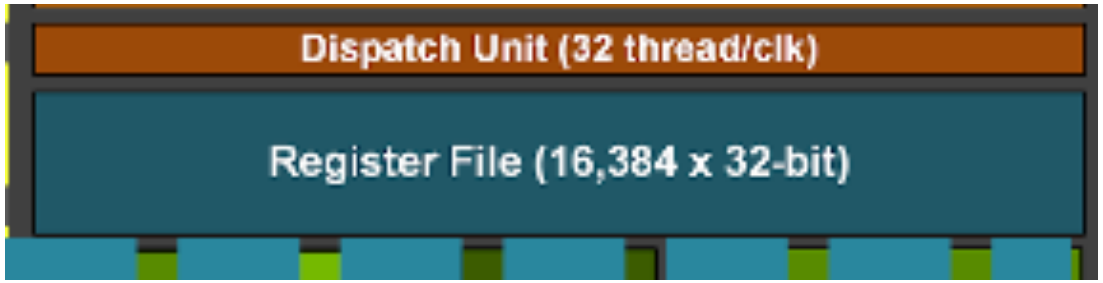
```
3.08 ms ± 47.4 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Shared memory matrix multiply

- Here the threads in a block load data into cache once
- Threads utilize fast shared memory to perform calculation
- Since our block grid is 16x16, each data load is reused at least 32 times
- Maximised data reuse which translates to performance gains!

Registers

- Registers are like CPU ones
- Can be thought of as variables in your code
- Measure of code complexity
- GPUs have limited number of registers shared between threads
- Too many registers used will result in spilling into memory



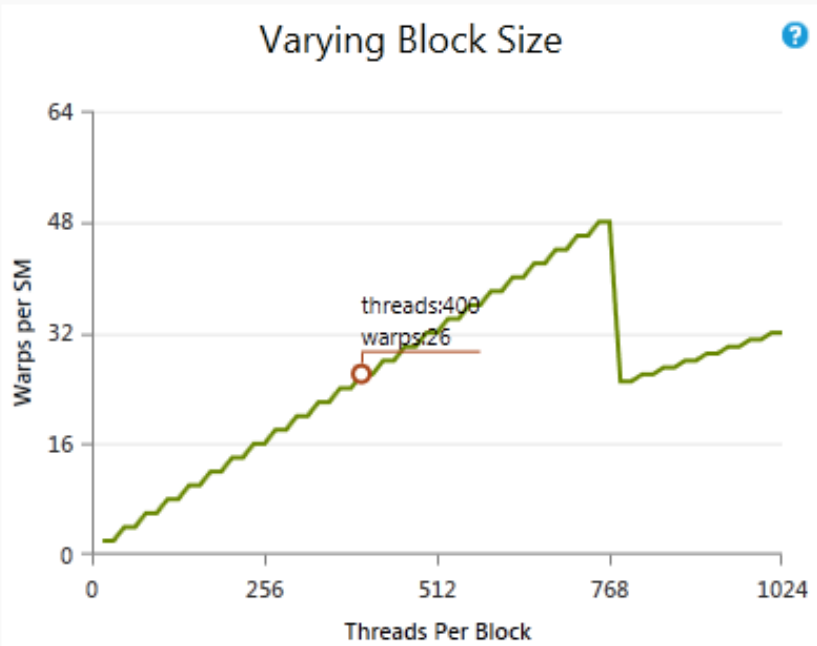
Registers

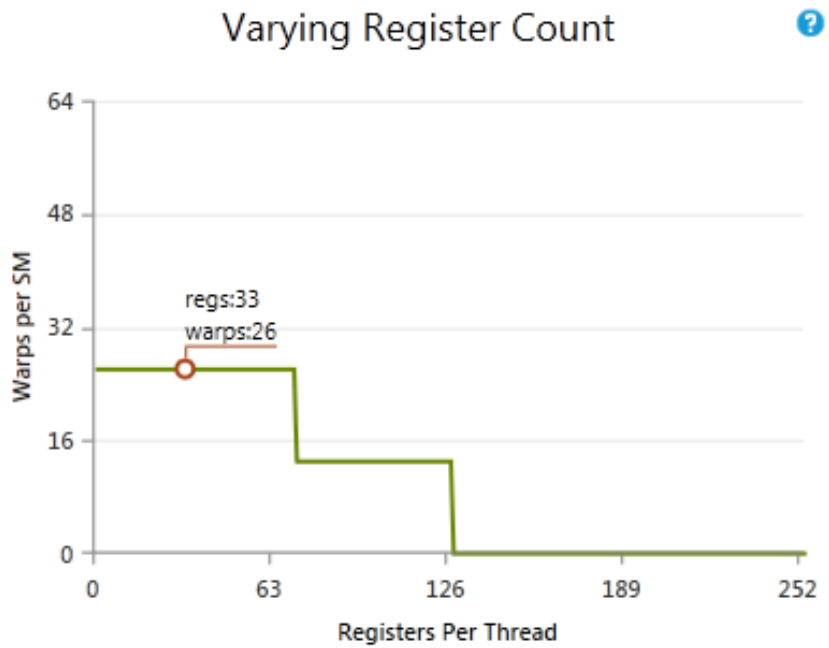
```
rbf_evaluation_cuda.get_regs_per_thread()
```

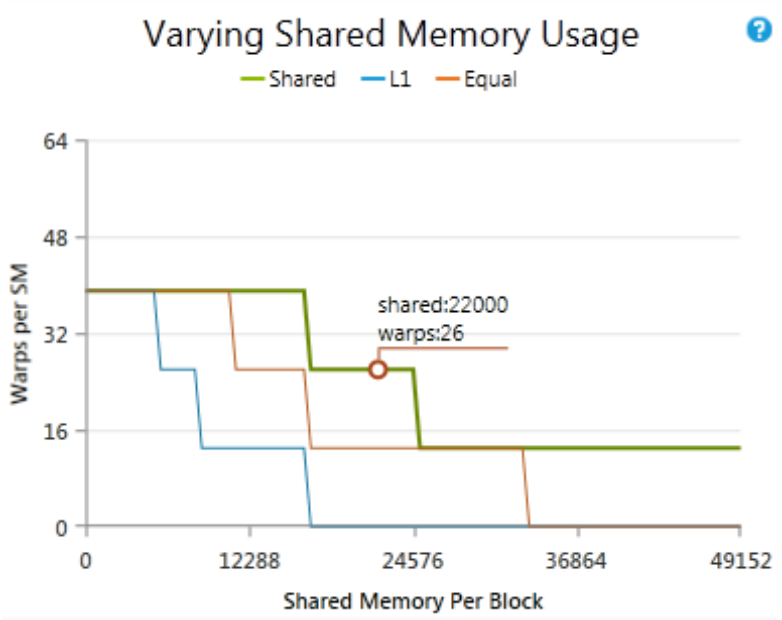
```
{(Array(float32, 2, 'C', False, aligned=True),
  Array(float32, 2, 'F', False, aligned=True),
  Array(float32, 1, 'C', False, aligned=True),
  Array(float32, 1, 'C', False, aligned=True)): 28}
```

Choice of block dimension

- The choice of threads per block depends on what resources are needed
- Shared memory and Registers are limited
- Less threads give more access to both registers and shared memory
- More threads means you can achieve greater usage the GPU
- “Occupancy”







CUDA Occupancy Calculator